Proyecto Final 1- Anima tu Estructura de Datos

Prof. Víctor Racsó Galván Oyola (vgalvan@utec.edu.pe)

Anima tu Estructura de Datos

1. Descripción

En este proyecto deberán elaborar un video educativo, al estilo de **Manim**[1] (Mathematical Animation Engine), que explique y demuestre visualmente el funcionamiento de una estructura de datos implementada en C/C++. El lenguaje de programación y la herramienta de animación son de **libre elección del grupo**: no es obligatorio usar Python ni Manim específicamente. Pueden usar la combinación que prefieran (por ejemplo, Manim en Python, Motion Canvas en TypeScript, Processing, o cualquier otra librería de animación en el lenguaje que elijan), siempre que el resultado cumpla con el contenido mínimo solicitado. Se recomienda revisar los videos del canal 3Blue1Brown[2] como referencia de estilo e inspiración.

2. Estructura de datos asignada

A cada grupo se le asignará, **de forma aleatoria**, una única estructura de datos según lo efectivamente cubierto en clase al momento de asignar el proyecto (**el docente confirmará y/o ajustará esta lista** según corresponda); no se repetirá una misma estructura entre grupos de una misma sección:

- **Monotonic Queue/Stack**: pila o cola cuyos elementos se mantienen en orden estrictamente creciente o decreciente, ideal para algoritmos de ventana deslizante (*sliding window*).
- **Associative Queue**: cola que permite las operaciones usuales de cola y una operación extra que implica calcular una función sobre los elementos de la cola en el orden de aparición (desde el `front` hasta el `back`).
- **LRU Cache**: combinación de una Tabla Hash y una Lista Doblemente Enlazada para acceder y expirar elementos en $O(1)$ según su orden de uso.
- **Red-Black Tree**: Árbol Binario de Búsqueda (BST) auto-balanceado mediante coloreado de nodos y rotaciones.

[1] `https://docs.manim.community/en/stable/index.html`
[2] `https://www.3blue1brown.com/`

- **Suffix Tree**: árbol comprimido de prefijos (Trie) que almacena todos los sufijos de una cadena para búsqueda rápida de patrones en texto.
- **Suffix Array**: arreglo de las posiciones de inicio de todos los sufijos de una cadena S ordenados lexicográficamente entre sí.
- **Quadtree**: árbol de partición espacial bidimensional donde cada nodo se divide en 4 hijos; utilizado en gráficos 2D y detección de colisiones.
- **Octree**: estructura de partición espacial tridimensional donde cada nodo se divide en 8 hijos; usada en motores gráficos 3D y simulación física.
- **Fenwick Tree**: arreglo unidimensional que utiliza operaciones a nivel de bits (*Binary Indexed Tree*) para calcular sumas acumuladas y actualizar valores en $O(\log N)$.
- **Segment Tree**: árbol binario almacenado en un vector para realizar consultas de rango (sumas, mínimos, máximos) y modificaciones en $O(\log N)$.
- **Skip List**: lista doblemente enlazada con múltiples niveles de punteros que permite búsquedas, inserciones y eliminaciones en $O(\log N)$ de forma probabilística.
- **Trie**: árbol de prefijos donde cada nodo representa un carácter, optimizado para almacenar diccionarios y realizar autocompletado en tiempo $O(L)$.
- **Bloom Filter**: estructura probabilística basada en un vector de bits y funciones hash para verificar la pertenencia de un elemento a un conjunto en $O(1)$.
- **Sqrt Decomposition**: divide un arreglo de N elementos en bloques de tamaño $O(\sqrt{N})$, precalculando un resumen por bloque para responder consultas de rango y actualizaciones en $O(\sqrt{N})$.
- **Sparse Table**: estructuras estáticas (sin actualizaciones) que precalculan respuestas para todos los subrangos de longitud potencia de 2 en $O(N \log N)$, respondiendo consultas de rango en $O(1)$; la Sparse Table clásica exige que la operación sea idempotente (mín, máx, gcd), mientras que la Disjoint Sparse Table generaliza esto a cualquier operación asociativa (por ejemplo, suma).
- **Treap**: Árbol Binario de Búsqueda balanceado probabilísticamente, donde cada nodo tiene además una prioridad aleatoria que debe cumplir la propiedad de heap; combinar ambas propiedades (BST en las claves, heap en las prioridades) da una altura esperada $O(\log N)$ y permite operaciones de *split*/*merge* eficientes.
- **Disjoint Set Union (DSU)**: mantiene una partición de elementos en conjuntos disjuntos, soportando `find` (¿a qué conjunto pertenece un elemento?) y `union` (unir dos conjuntos) en tiempo casi constante amortizado $O(\alpha(N))$ mediante unión por rango/tamaño y compresión de caminos.

3. ¿Qué debe incluir el video?

Se sugiere una duración de 1 a 2 minutos; en cualquier caso, **no debe exceder los 5 minutos**. La narración puede ser por voz en off, texto en pantalla, o ambos. Debe contener como mínimo:

- Una introducción conceptual: qué es la estructura, qué Tipo de Dato Abstracto (TDA) representa y para qué se usa en la práctica.
- Una demostración animada de sus operaciones principales (inserción, eliminación, búsqueda o recorrido, según corresponda).
- Al menos un caso borde relevante (estructura vacía, un solo elemento, peor caso, etc.).
- Un breve análisis de la complejidad temporal de las operaciones mostradas.
- El título del proyecto y los nombres de los integrantes, al inicio del video o en los créditos finales.

Nota (la animación debe ser real, no simulada): la animación debe estar impulsada por una implementación real de la estructura, programada por el propio grupo —no se permite animar valores o pasos "a mano" sin lógica real detrás, ni usar una implementación ya existente de la estructura asignada provista por alguna librería estándar o de terceros (por ejemplo `heapq`, `std::stack`, `java.util.Stack`, si esa es justamente la estructura asignada).

4. Entrega

- Código fuente (en el lenguaje elegido) que genera la animación, alojado en un **repositorio de GitHub** con un archivo `README.md` que presente el proyecto, el software requerido y los pasos para compilar/ejecutar el código.
- Video renderizado en formato MPEG-4 (`.mp4`) y alta resolución; si excede el límite de tamaño de Gradescope, incluyan un enlace (YouTube no listado, Drive, etc.) dentro del informe.
- Un informe breve en PDF: carátula (curso, integrantes, estructura asignada), descripción de la estructura y TDA, herramientas usadas y pasos para reproducir el video, contribuciones de cada integrante, análisis de complejidad, evidencia (capturas o enlace) con el enlace al repositorio de GitHub y al video, y al menos 2 conclusiones. Asegúrense de que los enlaces funcionen correctamente.

La entrega es grupal, única, vía Gradescope (solo un integrante la sube), y se califica con una rúbrica publicada en Gradescope que evalúa implementación, animación, contenido explicativo e informe.

5. Modalidad y fecha límite

Trabajan en grupos de **hasta 3 integrantes**.
Tienen hasta el **27/09/2026, hasta las 20:00**, para entregar el proyecto.

Si se inspiran en un video o ejemplo de terceros, cítenlo explícitamente en el informe.